

ANÁLISE DE ENGENHARIA REVERSA

Repositório GitHub "Tools" — junior700eletro-png

25 de maio de 2026

SUMÁRIO

1. Resumo Executivo
2. Arquitetura Atual
3. Componentes Principais
4. Fluxo de Dados
5. Caminhos de Desenvolvimento Identificados
6. Pontos de Integração
7. Gaps e Oportunidades
8. Roadmap Recomendado
9. Tecnologias e Dependências
10. Próximos Passos Críticos

1. Resumo Executivo

O repositório **Tools** do usuário **junior700eletro-png** é um conjunto de scripts, agentes e utilitários voltados para automação de processos de desenvolvimento, integração de sistemas e processamento de dados. A análise revela um ecossistema maduro, porém fragmentado, com forte predominância de PowerShell (70,9%) e Python (18,1%), complementado por HTML, Batchfile, PLpgSQL e JavaScript.

O objetivo principal do projeto é criar uma plataforma de automação que conecte agentes autônomos (como **Agente_Independente** e **Projeto_Agente_Depuracao_Codigo**), interfaces externas (**Interface_Adapta**), APIs (**google_drive_fastapi_server**) e utilitários de manipulação de arquivos (**Copiar arquivos**, **Copiar_recriar_projetos**).

O status atual indica uma base de código funcional, porém com necessidade de documentação técnica, padronização de estruturas e implementação de testes automatizados. Os agentes estão em estágio parcial de integração, e a API FastAPI para Google Drive oferece um ponto de partida para serviços externos.

2. Arquitetura Atual

2.1 Pastas Principais e Funções

Pasta	Função Principal	Tecnologia
Agente_Independente	Agente autônomo para execução de tarefas sem supervisão contínua	PowerShell / Python
Copiar arquivos	Utilitários de cópia e sincronização de arquivos entre diretórios	PowerShell (batch)
Copiar_recriar_projetos	Recriação de projetos a partir de templates ou fontes	PowerShell
Interface_Adapta	Interface de integração com a plataforma Adapta ONE	HTML / JavaScript
PS1_tools	Módulo principal de scripts PowerShell (70,9% do código total)	PowerShell 5.0+
PY_tools	Scripts Python para tarefas complementares (18,1%)	Python 3.8+
Projeto_Agente_Depuracao_Codigo	Agente de depuração de código com capacidades de análise estática	PowerShell / Python
Script_Patches	Patches e correções aplicadas a scripts ou sistemas externos	PowerShell / Batch

TemporariaPerplexity	Integração temporária com a API Perplexity AI	Python
Tesseract-OCR	Processamento de imagem e reconhecimento óptico de caracteres	Python / Tesseract
Teste	Scripts de teste e validação de funcionalidades	PowerShell / Python
google_drive_fastapi_server	Servidor FastAPI para gerenciamento de arquivos do Google Drive	Python / FastAPI
projeto_exemplo	Projeto de exemplo para demonstração de conceitos	Misto

2.2 Composição por Linguagem

Linguagem	Percentual	Principais Usos
PowerShell	70,9%	Automação de tarefas de sistema, agentes, scripts de manutenção
Python	18,1%	APIs, processamento de dados, OCR, integração com serviços externos
HTML	8,3%	Interface web da Interface_Adapta e documentação
Batchfile	1,3%	Scripts de inicialização e compatibilidade com Windows legado
PLpgSQL	1,1%	Consultas e procedures em banco de dados PostgreSQL

JavaScript	0,3%	Funcionalidades front-end na Interface_Adapta
------------	------	---

2.3 Padrões Arquiteturais Observados

A estrutura segue um modelo **orientado a scripts**, com cada pasta representando um módulo funcional independente. Não há um framework único de orquestração; a comunicação entre componentes ocorre via chamadas diretas (execução de scripts, invocação de APIs REST) e arquivos de configuração locais. A ausência de um arquivo de manifesto central (como *docker-compose* ou *requirements.txt* consolidado) indica que as dependências são gerenciadas de forma ad-hoc.

3. Componentes Principais

3.1 Agentes

- **Agente_Independente:** Script que opera autonomamente, sem intervenção manual. Realiza tarefas como monitoramento de diretórios, execução de tarefas programadas e resposta a eventos.
- **Projeto_Agente_Depuracao_Codigo:** Agente especializado em depuração de código. Analisa logs, identifica erros comuns e sugere correções. Atualmente integra chamadas a ferramentas de análise estática.

3.2 Scripts de Automação (PS1_tools e PY_tools)

Os diretórios **PS1_tools** e **PY_tools** concentram a maior parte do código. Em **PS1_tools**, encontram-se funções para manipulação de arquivos, manipulação do registro do Windows, execução remota via WinRM e coleta de informações do sistema. Em **PY_tools**, destacam-se módulos para scraping, manipulação de planilhas e integração com APIs REST.

3.3 Interfaces (Interface_Adapta)

A **Interface_Adapta** é uma aplicação web HTML/JS que consome a API do Adapta ONE. Fornece um dashboard para orquestração de tarefas, visualização de status de agentes e execução de scripts. É o ponto de entrada para operadores humanos.

3.4 Utilitários (Copiar arquivos, Copiar_recriar_projetos, Script_Patches)

- **Copiar arquivos:** Scripts para backup e sincronização de pastas. Suporta filtros por extensão e data.
- **Copiar_recriar_projetos:** Utilitário que replica a estrutura de um projeto de referência, substituindo placeholders (ex.: nomes de cliente, datas).
- **Script_Patches:** Conjunto de correções que podem ser aplicadas a arquivos de configuração ou código fonte em produção.

3.5 Integrações (google_drive_fastapi_server, Tesseract-OCR, Temporaria Perple

- **google_drive_fastapi_server:** API REST construída com FastAPI que expõe endpoints para upload, download, listagem e busca de arquivos no Google Drive. Usa a API oficial do Google Drive (OAuth 2.0).
- **Tesseract-OCR:** Scripts Python que utilizam o Tesseract para extrair texto de imagens e PDFs. Inclui pré-processamento de imagem (redimensionamento, binarização).
- **Temporaria Perplexity:** Integração experimental com a API da Perplexity para consultas de IA generativa. Atualmente desativada ou em fase de prototipação.

3.6 Testes e Exemplos

- **Teste:** Scripts de validação funcional, testes de unidade simples (sem framework padronizado).
- **projeto_exemplo:** Um projeto fictício usado como base para testes de recriação e patches.

4. Fluxo de Dados

4.1 Entrada

Os dados entram no sistema por múltiplas vias:

- Arquivos locais ou em rede (via **Copiar arquivos** ou monitoramento de diretórios)
- Requisições HTTP para a API FastAPI (**google_drive_fastapi_server**)
- Chamadas da plataforma Adapta ONE (via **Interface_Adapta**)
- Imagens e documentos digitalizados (via **Tesseract-OCR**)

4.2 Processamento

Após a entrada, os dados percorrem diferentes pipelines dependendo do tipo:

- **Arquivos de código:** passam por **Agente_Depuracao_Codigo** para análise de erros, depois são encaminhados para **Copiar_recriar_projetos** quando necessário.
- **Arquivos de configuração ou dados:** são processados por scripts **PS1_tools** ou **PY_tools** (transformações, validações, extração).
- **Imagens:** passam pelo pipeline OCR (pré-processamento !' Tesseract!' pós-processamento) e o texto extraído é inserido em banco de dados ou enviado para a interface.
- **Requisições externas:** a API FastAPI autentica, valida e roteia para os scripts correspondentes.

4.3 Saída

Os resultados gerados podem ser:

- Arquivos copiados, movidos ou transformados
- Relatórios de depuração (logs, sugestões de correção)
- Texto extraído de imagens
- Respostas da API (JSON) para consumidores externos
- Ações orquestradas no Adapta ONE

4.4 Integrações Externas

| Adapta ONE | API REST | Orquestração de tarefas e dashboards | | Tesseract OCR | Biblioteca local | Reconhecimento de texto em imagens |

5. Caminhos de Desenvolvimento Identificados

5.1 Caminho 1: Automação com Agentes

Fluxo: `Agente_Independente !' Projeto_Agente_Depuracao_Codigo !' Executar`

Status: Parcialmente implementado. O agente independente funciona, mas a comunicação com o agente de depuração é feita por script único, sem mensageria ou fila.

Próximos passos: Implementar uma fila de tarefas (RabbitMQ ou Redis), adicionar logging centralizado e criar um sistema de heartbeat para monitoramento dos agentes.

5.2 Caminho 2: Processamento de Projetos

Fluxo: Copiar arquivos !' Copiar_recriar_projetos !' Validar

Status: Framework existente e funcional. Os scripts de cópia e recriação estão operacionais, mas a validação ainda é manual ou com scripts simples.

Próximos passos: Automatizar a validação com testes de integração, adicionar templates versionados e criar um pipeline de CI/CD para projetos gerados.

5.3 Caminho 3: Integração Adapta

Fluxo: Interface_Adapta !' Orquestração !' Execução

Status: A interface web está criada e se comunica com a API do Adapta ONE, mas a orquestração ainda depende de triggers manuais.

Próximos passos: Sincronização bidirecional de dados entre Adapta e os agentes, implementação de webhooks e criação de dashboards de status em tempo real.

5.4 Caminho 4: API e Serviços

Fluxo: google_drive_fastapi_server !' Endpoints !' Automação

Status: API básica funcional, com endpoints de CRUD para arquivos no Google Drive. Falta autenticação robusta (apenas OAuth 2.0 básico) e versionamento de schema.

Próximos passos: Expandir funcionalidades (compartilhamento, pesquisa full-text, sincronização incremental), adicionar autenticação JWT e documentar via OpenAPI/Swagger.

6. Pontos de Integração

O repositório apresenta quatro pontos de integração com sistemas externos, cada um com maturidade distinta:

Google Drive (FastAPI server): Integração mais madura. Endpoints REST documentados no código. Usa *google-authgoogle-api-python-client*. Permite upload, download, listagem e exclusão.

Adapta ONE: Integração via API REST. A Interface_Adapta consome dados de tarefas e envia comandos. Falta tratamento de erros e rate limiting.

Tesseract OCR: Biblioteca local instalada no ambiente. Scripts Python chamam o executável via subprocess. A integração é direta, mas sem gerenciamento de dependências via virtualenv.

Perplexity AI: Integração temporária, com código comentado e sem chaves de API ativas no repositório. Provavelmente um teste conceitual.

7. Gaps e Oportunidades

A análise identifica os seguintes gaps que representam oportunidades de melhoria:

1. **Documentação técnica detalhada**— Ausente para a maioria dos módulos. Não há READMEs, diagramas de arquitetura ou guias de uso.
2. **Testes automatizados**— Apenas scripts de teste manuais na pasta Teste. Sem cobertura de testes unitários ou de integração.
3. **CI/CD pipeline**— Nenhum arquivo de configuração para GitHub Actions, Jenkins ou outra ferramenta de integração contínua.
4. **Logging centralizado**— Cada script utiliza logging próprio (Write-Host no PowerShell, print no Python), sem padrão ou agregação.
5. **Versionamento de schemas**— APIs (FastAPI) e integrações externas não possuem controle de versão explícito.
6. **Monitoramento em produção**— Ausência de métricas, alertas ou dashboards de saúde dos agentes e serviços.

"A oportunidade mais imediata é a criação de documentação mínima por módulo, que reduziria o atrito de onboarding e facilitaria a evolução paralela dos caminhos de desenvolvimento."

8. Roadmap Recomendado

8.1 Fase 1: Consolidação (Curto Prazo — 1 a 2 meses)

- Documentar a arquitetura de cada módulo (diagramas simples, README)
- Padronizar a estrutura de pastas e nomenclatura de scripts
- Implementar testes unitários para os módulos PS1_tools e PY_tools
- Adicionar logging centralizado com níveis (INFO, WARN, ERROR)
- Criar um arquivo `requirements.txt` e `manifest.psd1` para dependências

8.2 Fase 2: Integração (Médio Prazo — 3 a 6 meses)

- Conectar os agentes via fila de mensagens (RabbitMQ ou Redis)

- Implementar orquestração completa entre Interface_Adapta e agentes
- Sincronizar dados com Adapta ONE de forma bidirecional
- Expandir a API FastAPI com autenticação JWT e documentação Swagger

8.3 Fase 3: Automação (Longo Prazo — 6 a 12 meses)

- Configurar pipeline CI/CD (GitHub Actions) para testes e deploy
- Implementar monitoramento com Prometheus + Grafana ou ELK Stack
- Escalar a arquitetura para ambientes containerizados (Docker, Kubernetes)
- Criar playbooks de recuperação e disaster recovery

9. Tecnologias e Dependências

Tecnologia	Versão Mínima	Uso Principal
PowerShell	5.0+	Automação principal, agentes, utilitários
Python	3.8+	APIs, OCR, integrações externas
FastAPI	0.68+	Servidor REST para Google Drive
Google Drive API	v3	Autenticação e manipulação de arquivos
Tesseract OCR	4.x	Reconhecimento de texto em imagens
Adapta ONE API	—	Interface de orquestração de tarefas
PostgreSQL (PLpgSQL)	12+	Procedures armazenadas no banco

Node.js (opcional)	—	Execução de scripts HTML/JS da interface
--------------------	---	--

10. Próximos Passos Críticos

1. **Mapear dependências exatas**de cada script — criar arquivos de manifesto (requirements.txt, manifest.psd1) para reprodutibilidade.
2. **Criar documentação de APIs**— pelo menos um endpoint de teste no FastAPI com descrição de parâmetros e respostas.
3. **Implementar testes unitários**para os módulos PS1_tools e PY_tools usando Pester (PowerShell) e pytest (Python).
4. **Estabelecer padrões de código**— linters (PSScriptAnalyzer, flake8) e um guia de estilo.
5. **Configurar CI/CD**no GitHub Actions com execução de testes, linting e geração de documentação automática.